Salesforce Enterprise DX & Packaging Handbook

From building Apex/LWC applications to testing, packaging, installation, release, GitHub integration, data persistence, and enterprise hardening

Based on the Task Manager / ToDO_App_Project workflow completed on 8 October 2026

This is deliberately broader than the original assignment. It reconstructs the work performed, explains why each step was performed, and turns it into a reusable workflow for future Salesforce applications.

Current project reference: ToDO_App_Project
Application: Task Manager
Dev Hub: FallingLily
Development scratch org: Selection_Project
Package type actually used: Namespace-free Unlocked Package
Package ID: 0Hog70000003qZxCAI
Subscriber Package Version: 04tg7000000WBoLAAW

Contents

1. What packaging is and why it matters
2. Package-type decision: Unlocked, Managed 2GP, legacy Unmanaged/Managed 1GP
3. Salesforce DX project architecture
4. Complete application-building workflow
5. Apex class and test-class workflow
6. LWC workflow: HTML, JavaScript, metadata and Apex
7. Data model and DML: making the application actually persist data
8. Deployment, retrieval and scratch-org lifecycle
9. Namespace strategy
10. Unlocked Package: complete reusable CLI workflow
11. Managed 2GP: complete reusable CLI workflow
12. Legacy first-generation unmanaged package workflow
13. Legacy first-generation managed package workflow
14. Installation and clean-org testing
15. Versioning, promotion, dependencies and upgrades
16. GitHub/source control workflow
17. Enterprise-grade security and configuration
18. CI/CD and release pipeline
19. Packaging checklist and failure diagnosis
20. Improvements to Task Manager and lessons learned
21. Enterprise project blueprint
22. Official Salesforce references

1. What Salesforce packaging actually does

A Salesforce package is a distributable container for metadata and related application components. A package version is a specific, installable snapshot of that source. This distinction is fundamental: your local folder is source; the package version is the release artifact installed into another org.

For an enterprise application, the goal is not merely to make a package install. The goal is to make the installed org behave correctly without depending on the developer's scratch org, developer-only records, hard-coded IDs, undocumented configuration, or missing permissions.

A useful mental model is:

```
GitHub
  ■
  ▼
Salesforce DX source
  ■
  ■■■ Apex
  ■■■ LWC
  ■■■ Objects / Fields
  ■■■ Permission Sets
  ■■■ Flows / Validation / Custom Metadata
  ■■■ App / Tabs / Assets
  ■■■ Tests
  ■
  ▼
Scratch org development
  ■
  ▼
Deploy + automated tests
  ■
  ▼
Package version
  ■
  ▼
Clean subscriber/test org
  ■
  ▼
Release / promote
  ■
  ▼
Production / customer orgs
```

Salesforce's DX model is source-driven: the project and version-control system should be the source of truth rather than treating an org as the canonical copy. Salesforce explicitly recommends storing DX project contents in source control. ■cite■turn0search5■turn0search6■

2. Package-type decision

For a new enterprise project, distinguish the modern package types from legacy first-generation terminology.

Type	Use	Namespace	Modern recommendation
Unlocked Package	Internal apps, modular enterprise deployment, admin-managed configuration	Optional	Default choice for most internal apps
Managed 2GP	ISV/customer distribution, AppExchange-style product	Required	Use for distributable commercial apps
Unmanaged 1GP	Legacy metadata bundle	No managed ownership model	Use mainly when required by an existing legacy proce
Managed 1GP	Legacy managed package	Namespace required	Maintain/extend existing 1GP only when necessary

Salesforce's current guidance describes unlocked packages as especially suited to internal business applications, while managed 2GP is the distribution model for apps intended for AppExchange/customer distribution. ■cite■turn0search6■turn0search8■turn1search4■

Important terminology correction: there is no modern **sf package create -t Unmanaged** workflow. The current **sf package** commands cover Unlocked and Managed 2GP packages. Legacy first-generation managed/unmanaged packages use the **sf package1** command family. Salesforce's CLI reference explicitly describes package1 as the command family for first-generation managed and unmanaged packages. ■cite■turn2search0■turn2search1■

3. Salesforce DX project architecture

A standard DX project has an sfdx-project.json file at its root and normally places application metadata under force-app/main/default. Salesforce source format decomposes metadata into files and directories that are easier to version-control and merge. LWC bundles live under the lwc directory. ■cite■turn0search3■turn1search3■

```
ToDO_App_Project/
■■■ sfdx-project.json
■■■ config/
■   ■■■ project-scratch-def.json
■■■ force-app/
■   ■■■ main/
■       ■■■ default/
■           ■■■ applications/
■           ■■■ classes/
■           ■■■ contentassets/
■           ■■■ customMetadata/
■           ■■■ flows/
■           ■■■ lwc/
■           ■   ■■■ taskManager/
■           ■■■ objects/
■           ■■■ permissionsets/
■           ■■■ tabs/
■           ■■■ ...
■■■ scripts/
■■■ README.md
■■■ .gitignore
```

Keep packageable metadata inside the package directory. Avoid committing secrets, authentication tokens, scratch-org credentials, generated files, or local environment state.

4. Start a new project

For a new project, authorize the Dev Hub and create a DX project before developing the application.

```
sf --version
sf project generate --name EnterpriseTaskManager
cd EnterpriseTaskManager

# Authorize the Dev Hub interactively
sf org login web --alias FallingLily --set-default-dev-hub

# Inspect authenticated orgs
sf org list
```

The exact project-generation flags can vary slightly with Salesforce CLI releases, so use **sf project generate --help** if your installed CLI exposes different flag names.

5. Create the application components

An enterprise application is not just an LWC. The UI, server-side logic, schema, security model, automation, configuration, tests, and packaging metadata must work together.

Apex class:

```
sf apex generate class --name TaskManagerController --output-dir force-app/main/default/classes
```

Apex test class:

```
sf apex generate class --name TaskManagerControllerTest --output-dir force-app/main/default/classes
```

The test class should be annotated with @IsTest and should test behavior, not merely execute lines for coverage.

LWC:

```
sf lightning generate component --name taskManager --type lwc --output-dir force-app/main/default/lwc
```

The generated bundle normally contains HTML, JavaScript, and the component metadata XML. The metadata XML determines where the component can be exposed, such as an app page or record page.

6. Apex controller design

For a simple Task Manager, the Apex controller should expose a small, deliberate API to the LWC. For example: getTasks(), createTask(), updateTask(), and deleteTask().

```
public with sharing class TaskManagerController {
    @AuraEnabled(cacheable=true)
```

```
    public static List<Task__c> getTasks() {
        return [
            SELECT Id, Name, Status__c, Due_Date__c
            FROM Task__c
            ORDER BY Due_Date__c ASC
        ];
    }

    @AuraEnabled
    public static Task__c createTask(String name, String status) {
        Task__c task = new Task__c(
            Name = name,
            Status__c = status
        );
        insert task;
        return task;
    }

    @AuraEnabled
    public static Task__c updateTask(Id taskId, String status) {
        Task__c task = [
            SELECT Id, Status__c
            FROM Task__c
            WHERE Id = :taskId
            LIMIT 1
        ];
        task.Status__c = status;
        update task;
        return task;
    }

    @AuraEnabled
    public static void deleteTask(Id taskId) {
        delete [SELECT Id FROM Task__c WHERE Id = :taskId LIMIT 1];
    }
}
```

For production-quality code, add CRUD/FLS enforcement appropriate to the application's architecture, validate inputs, handle expected exceptions, avoid SOQL/DML in loops, and bulkify methods that can receive collections. The sample above is intentionally compact for learning.

7. Apex test strategy

A good test class creates its own test data and does not depend on records already existing in the scratch org. This is one of the central lessons missing from the original Task Manager packaging exercise.

```
@IsTest
private class TaskManagerControllerTest {

    @TestSetup
    static void setupData() {
        insert new Task__c(
            Name = 'Initial Task',
            Status__c = 'Open'
        );
    }

    @IsTest
    static void testGetTasks() {
        Test.startTest();
        List<Task__c> tasks = TaskManagerController.getTasks();
        Test.stopTest();

        System.assertEquals(1, tasks.size());
        System.assertEquals('Initial Task', tasks[0].Name);
    }

    @IsTest
    static void testCreateTask() {
        Test.startTest();
        Task__c task =
            TaskManagerController.createTask('New Task', 'Open');
        Test.stopTest();

        System.assertNotEquals(null, task.Id);
    }

    @IsTest
    static void testUpdateTask() {
```

```
        Task__c task = [SELECT Id FROM Task__c LIMIT 1];

        Test.startTest();
        TaskManagerController.updateTask(task.Id, 'Done');
        Test.stopTest();

        Task__c updated = [
            SELECT Status__c FROM Task__c WHERE Id = :task.Id
        ];
        System.assertEquals('Done', updated.Status__c);
    }

    @IsTest
    static void testDeleteTask() {
        Task__c task = new Task__c(
            Name = 'Delete Me',
            Status__c = 'Open'
        );
        insert task;

        Test.startTest();
        TaskManagerController.deleteTask(task.Id);
        Test.stopTest();

        System.assertEquals(
            0,
            [SELECT COUNT() FROM Task__c WHERE Id = :task.Id]
        );
    }
}
```

The four tests in the original project—create, delete, get, and update—passed, and the controller and org-wide coverage were reported at 100%. That validated the controller code, but it did not by itself prove that a newly installed org had the required data model, permissions, configuration, and persisted records.

8. LWC ↔ Apex data flow

The LWC should treat Apex as the server boundary. A common pattern is: component loads → calls Apex → receives records → renders them → user edits → component calls Apex DML method → server returns the saved record → UI refreshes.

```
import { LightningElement, wire } from 'lwc';
import getTasks from '@salesforce/apex/TaskManagerController.getTasks';
import createTask from '@salesforce/apex/TaskManagerController.createTask';

export default class TaskManager extends LightningElement {
    tasks = [];
    error;

    @wire(getTasks)
    wiredTasks({ data, error }) {
        if (data) {
            this.tasks = data;
            this.error = undefined;
        } else if (error) {
            this.error = error;
            this.tasks = [];
        }
    }

    async handleCreate() {
        try {
            await createTask({
                name: 'New Task',
                status: 'Open'
            });
            // Refresh strategy should be added here.
        } catch (e) {
            this.error = e;
        }
    }
}
```

For real applications, use Lightning Data Service where appropriate, refresh wired data after mutations, surface meaningful errors, disable duplicate submissions, validate form input, and avoid unnecessary Apex calls.

9. Data model: the major improvement to Task Manager

The original Task Manager could demonstrate controller operations but did not establish a complete persistent data model in the package. That is the largest functional improvement to make before treating the project as a serious application.

Create a custom object such as **Task__c** with fields such as Name, Status__c, Priority__c, Due_Date__c, Description__c, Owner/Assignee, and optionally a relationship to the application's parent business object.

```
force-app/main/default/objects/Task__c/
■■■ Task__c.object-meta.xml
■■■ fields/
    ■■■ Status__c.field-meta.xml
    ■■■ Priority__c.field-meta.xml
    ■■■ Due_Date__c.field-meta.xml
    ■■■ Description__c.field-meta.xml
```

The package must contain the schema required by the application. The application should create and manipulate records at runtime through DML; the package should not rely on records that happened to exist in the developer's scratch org.

This distinction is critical:

```
METADATA = structure
  Object
  Fields
  Permission Sets
  Apex
  LWC
  Flows
  Custom Metadata

DATA = runtime records
  Task__c records
  User-created business records
  Transactions
  Customer data
```

Normal package installation does not mean your developer-org business records magically become application data in every subscriber org. For required seed/configuration data, consider Custom Metadata Types for configuration, package-install/post-install logic where appropriate, or a controlled setup wizard. For user/business data, create records through the application itself.

10. Make the application portable across orgs

A package is broken if the UI assumes a record ID from the developer org, a user ID, a Record Type ID, a hard-coded Profile ID, or an external endpoint that exists only in one environment.

Use stable identifiers and configuration mechanisms. Examples: Custom Metadata for business configuration, DeveloperName rather than hard-coded record IDs where supported, Permission Sets for access, Named Credentials for external integrations, and Custom Labels for translatable text.

For records that must be created in multiple orgs, use runtime creation or a setup process. If importing known records, use External IDs and upsert semantics rather than copying Salesforce IDs between orgs.

11. Deploy and retrieve source

During development, deployment synchronizes source to an org. Retrieval brings org metadata into source format. The exact command should target the appropriate alias so that you never accidentally deploy experimental code to the wrong org.

```
# Deploy the project
sf project deploy start --source-dir force-app --target-org Selection_Project

# Run Apex tests as part of deployment
sf project deploy start --source-dir force-app --target-org Selection_Project   --test-level RunLocalTests

# Retrieve source from an org
sf project retrieve start --target-org Selection_Project --source-dir force-app

# Open the org
sf org open --target-org Selection_Project
```

In the original project, deployment successfully changed the Task_Manager CustomApplication. A later 'No changes to deploy' result was normal because the source and org were already synchronized.

12. Scratch-org lifecycle

Use a development scratch org for implementation and a separate clean scratch org for package installation testing. This catches dependencies that the development org masks.

```
# Create a development scratch org
sf org create scratch   --definition-file config/project-scratch-def.json   --alias Selection_Project   --duration-days 7   --tar

# Create a clean package-test org
sf org create scratch   --definition-file config/project-scratch-def.json   --alias PackageTest   --duration-days 1   --target-de
```

Salesforce's packaging workflows explicitly recommend installing the package into a different scratch org from the one used for development. ■cite■turn1search1■

13. Namespace strategy

A namespace is a 1–15 character identifier that distinguishes a package and its contents. When a namespace is used, package component API names receive the namespace prefix. Salesforce allows unlocked packages with or without a namespace; the namespace is assigned when the package is created and cannot simply be changed later. ■cite■turn0search2■turn0search4■

```
No namespace:
Task__c

Namespace "acme":
acme__Task__c
```

For the original project, removing Ajher17 was correct because the trainer's fallback explicitly allowed a namespace-free Unlocked Package. For an enterprise internal application, a no-namespace unlocked package is often simpler. For a product intended for external distribution, use a deliberate namespace strategy and Managed 2GP.

Important compatibility point: Salesforce documents that a no-namespace unlocked package cannot be installed into an org that already has a namespace. Therefore, check the target org's namespace situation before using a no-namespace package as a universal artifact. ■cite■turn0search9■turn0search11■

14. Complete Unlocked Package workflow

This is the modern workflow to memorize for internal/enterprise applications.

Step 1 — Prepare Dev Hub and source control.

```
sf org login web --alias FallingLily --set-default-dev-hub
sf org list
```

Step 2 — Build and test in a scratch org.

```
sf org create scratch --definition-file config/project-scratch-def.json   --alias DevOrg --duration-days 7 --target-dev-hub Falli

sf project deploy start --source-dir force-app --target-org DevOrg
sf apex run test --target-org DevOrg --test-level RunLocalTests
```

Step 3 — Configure packageDirectories in sfdx-project.json.

```
{
  "packageDirectories": [{
    "path": "force-app",
    "default": true,
    "package": "EnterpriseTaskManager",
    "versionName": "Version 1.0",
    "versionNumber": "1.0.0.NEXT",
    "versionDescription": "Enterprise Task Manager"
  }],
  "name": "EnterpriseTaskManager",
  "sfdcLoginUrl": "https://login.salesforce.com",
  "sourceApiVersion": "67.0"
}
```

Step 4 — Create the package. For a namespace-free unlocked package:

```
sf package create   --name "EnterpriseTaskManager"   --package-type Unlocked   --path force-app   --target-dev-hub FallingLily
```

The package name becomes a useful alias and the CLI can update sfdx-project.json with package information. Salesforce recommends package aliases because they are easier to use in scripts than raw IDs. ■cite■turn1search0■turn1search5■

Step 5 — Verify the package.

```
sf package list --target-dev-hub FallingLily
sf package list --target-dev-hub FallingLily --verbose
```

Step 6 — Create a package version.

```
sf package version create   --package "EnterpriseTaskManager"   --code-coverage   --wait 30   --target-dev-hub FallingLily   --ta
```

Use versionNumber 1.0.0.NEXT or explicitly supply a version number. Package versions are immutable snapshots. Salesforce recommends keeping package version tags synchronized with version control. ■cite■turn1search6■turn0search11■

Step 7 — Monitor creation.

```
sf package version create list --target-dev-hub FallingLily
sf package version create report --package-create-request-id 08c...   --target-dev-hub FallingLily
```

Step 8 — List versions and obtain the 04t ID.

```
sf package version list   --packages "EnterpriseTaskManager"   --target-dev-hub FallingLily
```

A Subscriber Package Version ID starts with **04t** and is the ID used for installation. Package IDs begin with 0Ho and version-creation requests begin with 08c. ■cite■turn1search5■

Step 9 — Install into a clean org.

```
sf package install   --package 04tXXXXXXXXXXXXXX   --target-org PackageTest   --wait 30   --publish-wait 30   --security-type Ad
```

Step 10 — Test the installed artifact, not just the source.

```
sf org open --target-org PackageTest
sf package installed list --target-org PackageTest
```

Step 11 — Promote only after testing.

```
sf package version promote   --package 04tXXXXXXXXXXXXXX   --target-dev-hub FallingLily   --no-prompt
```

Salesforce states that newly created managed 2GP versions are beta and that promotion is a one-time, irreversible release transition. For unlocked packages, promotion is also part of the package lifecycle when you want a released version. ■cite■turn1search8■turn0search18■

15. Managed 2GP workflow

Managed 2GP is the modern managed packaging path and is the appropriate route for an application intended for external/customer distribution or AppExchange. Salesforce recommends 2GP for new managed packages. ■cite■turn1search9■turn1search4■

Prerequisite: the namespace must be created and linked to the Dev Hub. Salesforce states that a namespace is required for managed 2GP. ■cite■turn1search11■turn0search4■

Step 1 — Register/link the namespace.

```
# In the namespace Developer Edition org:
# Setup → Package Manager → Namespace Settings → create namespace

# In the Dev Hub:
# Setup → Namespace Registry → link the namespace

# Then configure the project:
{
  "namespace": "acme"
}
```

Step 2 — Create the managed package.

```
sf package create   --name "EnterpriseTaskManager"   --package-type Managed   --path force-app   --target-dev-hub FallingLily
```

Do not attempt to change the package type or namespace after creation. Salesforce documents both as package-creation decisions that cannot simply be changed later. ■cite■turn1search0■

Step 3 — Create a version.

```
sf package version create   --package "EnterpriseTaskManager"   --code-coverage   --wait 30   --target-dev-hub FallingLily   --ta
```

For later managed 2GP versions, configure package ancestry appropriately. Salesforce requires the package ancestor to be the highest promoted version for that package. ■cite■turn1search2■

Step 4 — Install in a clean org, test, then promote.

```
sf package install --package 04tXXXXXXXXXXXXXX   --target-org PackageTest --wait 30

sf package version promote   --package 04tXXXXXXXXXXXXXX   --target-dev-hub FallingLily   --no-prompt
```

Managed packages have stricter component manageability and upgrade semantics. Design the public API, namespace, metadata visibility, and upgrade path before releasing the first production version.

16. Legacy first-generation unmanaged package workflow

Unmanaged packages are a legacy first-generation packaging model. They are not created with the modern **sf package create -t Unlocked** or **-t Managed** commands. Salesforce retains the **sf package1** command family for first-generation managed and unmanaged packages. ■cite■turn2search0■

The package itself is created/configured through the first-generation Package Manager/packaging-org model. Once you have a first-generation package ID (033...), the CLI can create a version from it.

For an unmanaged version, omit --managed-released.

```
sf package1 version create   --package-id 033XXXXXXXXXXXXXX   --name "Version 1.0"   --version 1.0   --target-org PackagingOrg
```

Salesforce's CLI reference explicitly states that omitting --managed-released creates an unmanaged package version; adding that flag creates a managed released package version. ■cite■turn2search1■

List and install:

```
sf package1 version list --package-id 033XXXXXXXXXXXXXX   --target-org PackagingOrg

sf package install   --package 04tXXXXXXXXXXXXXX   --target-org TestOrg   --wait 30
```

Use unmanaged packages mainly when an existing legacy process requires them. For a new enterprise project, prefer an unlocked package unless there is a specific reason to use managed 2GP.

17. Legacy first-generation managed package workflow

First-generation managed packaging is also legacy for new development. It requires a namespace and follows a different packaging-org workflow. Salesforce's current guidance recommends 2GP for new managed packages, while 1GP remains relevant to existing packages and certain legacy integrations. ■cite■turn1search4■turn2search6■

```
# Package version creation
sf package1 version create   --package-id 033XXXXXXXXXXXXXX   --name "Version 1.0"   --version 1.0   --target-org PackagingOrg

# List versions
sf package1 version list --package-id 033XXXXXXXXXXXXXX   --target-org PackagingOrg

# Install a 04t version
sf package install --package 04tXXXXXXXXXXXXXX   --target-org TestOrg --wait 30
```

The important lesson is not to mix command families: **sf package** is the modern 2GP/unlocked family; **sf package1** is the first-generation family.

18. Installation and clean-org validation

A clean installation is one of the most valuable tests you can perform. Your development org can contain manual metadata, records, permission assignments, connected apps, custom settings, and other artifacts that accidentally hide package dependencies.

```
sf org list

sf package install   --package 04tg7000000WBoLAAW   --target-org PackageTest   --wait 30   --security-type AdminsOnly

sf package installed list --target-org PackageTest
sf org open --target-org PackageTest
```

You can also use the generated browser installation URL for a package version. Be careful to verify which Salesforce org is logged into the browser before installing.

Test at least: application opens; navigation tab exists; LWC renders; Apex calls succeed; create/read/update/delete operations work; permissions are correct; configuration is present; no developer-org IDs are assumed; and the package does not require undocumented manual setup.

19. Versioning and upgrade discipline

Treat every package version as an immutable release artifact. Keep the version number in sfdx-project.json meaningful and use NEXT for unique build numbers when appropriate. Salesforce warns that repeatedly creating the same MAJOR.MINOR.PATCH.BUILD number is undesirable and documents version-number rules for 2GP. ■cite■turn1search15■

```
1.0.0.NEXT  → development/build
```

```
1.0.1.NEXT  → bug fix
1.1.0.NEXT  → feature release
2.0.0.NEXT  → breaking/major release
```

For managed 2GP, ancestry matters because Salesforce uses the ancestor relationship to understand upgrades. For unlocked packages, think about upgrade behavior too: removed components can be deprecated or deleted depending on the chosen upgrade strategy. The package install command documents Mixed, DeprecateOnly, and Delete upgrade behaviors. ■cite■turn0search12■

20. Dependencies

Enterprise apps frequently depend on other packages or platform capabilities. Explicitly model dependencies rather than assuming the target org happens to contain them.

```
Example conceptual dependency graph:

Core Security Package
        ■
        ■■■ Shared Objects / Custom Metadata
        ■
        ▼
Task Manager Package
        ■
        ■■■ Apex
        ■■■ LWC
        ■■■ Flows
        ■
        ▼
Optional Integration Package
```

Use package dependencies where appropriate, and keep modular packages cohesive. Salesforce documents dependencies as part of unlocked and managed 2GP packaging. ■cite■turn1search11■turn0search19■

21. GitHub: what was missing from this project

The original assignment did not utilize GitHub. For an enterprise application, that is a significant process gap. Salesforce DX is designed around source control; the source repository should be the authoritative record of the application.

Basic setup:

```
cd D:\ToDO_App_Project

git init
git branch -M main
git status

git add .
git commit -m "Initial Task Manager application"

git remote add origin git@github.com:YOUR-USER/YOUR-REPO.git
git push -u origin main
```

For future work, use feature branches:

```
git checkout -b feature/task-persistence
# make changes
git add .
git commit -m "Add persistent task records"
git push -u origin feature/task-persistence
```

Then create a pull request and require review before merging. Package version creation should occur from a known commit/tag, not from an arbitrary developer workstation state.

Salesforce specifically recommends storing the DX project and tests in source control, and unlocked package guidance recommends using version-control tags with package versions. ■cite■turn0search5■turn0search11■

22. Recommended repository layout

```
EnterpriseTaskManager/
■■■ .gitignore
■■■ README.md
■■■ sfdx-project.json
■■■ config/
■   ■■■ project-scratch-def.json
```

```
■■■ force-app/
■   ■■■ main/default/
■■■ scripts/
■   ■■■ setup/
■   ■■■ ci/
■■■ docs/
■   ■■■ architecture.md
■   ■■■ deployment.md
■   ■■■ release-notes/
■■■ .github/
    ■■■ workflows/
        ■■■ validate.yml
        ■■■ release.yml
```

The README should explain prerequisites, authorization, scratch-org creation, deployment, test execution, packaging, installation, configuration, and known limitations.

23. Enterprise security and configuration

Before calling an application enterprise-grade, address access control deliberately.

• Use with sharing or an explicit sharing strategy for Apex classes.

• Use Permission Sets rather than embedding broad permissions into a profile whenever practical.

• Package the permission sets required by the application.

• Enforce object- and field-level security for Apex that crosses user-controlled data boundaries.

• Validate user input on both client and server boundaries.

• Never hard-code Salesforce record IDs.

• Never store secrets, passwords, API keys, or tokens in source control.

• Use Named Credentials for external callouts rather than embedding endpoint credentials in Apex.

• Use Custom Metadata for deployable business configuration that should travel with the application.

• Use Custom Labels for user-facing text and translation support.

• Keep integration configuration environment-specific; do not assume the developer endpoint exists in production.

• Add negative tests for unauthorized access, invalid input, missing records, duplicate operations, and integration failures.

24. Integration and Named Credentials

If the enterprise application sends data to an external API, the package should contain the reusable integration logic and configuration metadata, but environment-specific credentials should be supplied through the target org's secure configuration.

```
Conceptual flow:

LWC
  ↓
Apex service
  ↓
Named Credential
  ↓
External API

Do NOT do:

LWC
  ↓
hard-coded API key / password / token
  ↓
External API
```

This separation allows the same package version to be installed into development, QA, UAT, and production while each environment supplies its own secure connection configuration.

25. CI/CD blueprint with GitHub

A useful enterprise pipeline is:

```
Pull Request
    ↓
Static checks / formatting
    ↓
Scratch org validation
    ↓
Deploy source
    ↓
Run Apex tests
    ↓
Package version / validation artifact
    ↓
Clean-org installation test
    ↓
Approval
    ↓
Release / promote
    ↓
Deploy or install to production
```

The important principle is that packaging becomes an automated build artifact rather than a manual command typed from a developer's laptop. Git commit/tag → package version → release is the traceability chain.

For production CI/CD, use secure authentication and protected repository secrets. Do not commit Salesforce session tokens, private keys, client secrets, or org credentials.

26. Task Manager: concrete improvements before an enterprise project

1. Add persistent Task__c records. The LWC should create/read/update/delete actual records through Apex DML.

2. Package the Task__c object and fields. The installed org must receive the schema required by the code.

3. Add Permission Set. Users of the application need object, field, Apex class, and tab access.

4. Remove all hard-coded IDs. Resolve records by stable configuration/developer names or create them dynamically.

5. Add test data creation. Tests must create their own data so a clean org is valid.

6. Add negative tests. Test missing records, invalid input, permissions, duplicate actions, and error paths.

7. Add Custom Metadata. Store configurable statuses, limits, labels, or feature switches in deployable metadata where appropriate.

8. Add GitHub. Track every change and tag every package release.

9. Add clean-org installation testing. Do not consider a package complete because it works in the development org.

10. Add release documentation. Include prerequisites, installation instructions, configuration, upgrade notes, and known limitations.

11. Add CI. Validate pull requests automatically before merging.

12. Add observability. Define useful logging/error reporting for production failures without exposing sensitive data.

27. Why DML matters in this particular project

You correctly identified a gap: merely having controller methods named create/update/delete is weaker than demonstrating that the application persists and retrieves records in a clean org. The application should have a complete round trip:

```
User enters task
      ↓
LWC state
      ↓
Apex @AuraEnabled method
      ↓
Validate input
      ↓
DML insert/update/delete
      ↓
Salesforce database
      ↓
SOQL query
      ↓
Apex response
```

```
      ↓
LWC state refresh
      ↓
User sees persisted task
```

That is the architecture to practice before the enterprise project. It demonstrates both the Salesforce data layer and the UI/server boundary rather than only packaging.

28. Packaging failure checklist

Symptom	Likely cause	Check
Package create fails	Invalid project/package configuration	sfdx-project.json, package type, namespace
SingleNonDefaultPackageError	Only package directory is not default	Set "default": true
InvalidIdOrAliasError	Package alias missing/incorrect	packageAliases / package list
Missing metadata during package build	Referenced component not in source	Retrieve component and include it
App references missing Utility Bar	Unsupported/missing metadata reference	Remove or package supported metadata
Install fails in clean org	Hidden dependency	Inspect missing metadata, permissions, features, packages
LWC loads but DML fails	Object/FLS/CRUD/permission issue	Permission Set + Apex security
Works only in developer org	Hard-coded IDs or org-specific config	Custom Metadata / runtime lookup
No records after installation	Confusing metadata with data	Implement runtime DML/setup seed strategy
Namespace installation failure	Target org/package namespace mismatch	Check namespace compatibility
Version creation still running	Long asynchronous build	package version create list/report
Upgrade breaks data	Unsafe destructive change	Review upgrade type and metadata dependencies

29. Exact reproduction of today's completed Task Manager workflow

The actual project used these important values and decisions:

```
Project:
D:\ToDO_App_Project

Dev Hub:
FallingLily

Development scratch org:
Selection_Project

Original namespace:
Ajher17  → removed

Final package:
Task Manager

Package type:
Unlocked

Namespace:
none

Package ID:
0Hog70000003qZxCAI

Version creation request:
08cg70000009h49AAA

Subscriber Package Version:
04tg7000000WBoLAAW

Installation URL:
https://login.salesforce.com/packaging/installPackage.apexp?p0=04tg7000000WBoLAAW
```

The project originally referenced a custom tab and utility bar. The custom tab was retrieved from Selection_Project. The UtilityBar reference was removed because the current CLI metadata registry did not support the direct retrieval path. The

X256x256 ContentAsset logo was retrieved and retained.

The Apex tests passed: testCreateTask, testDeleteTask, testGetTasks, and testUpdateTask. TaskManagerController coverage and reported org-wide coverage were both 100%.

Package creation and package version creation succeeded. The remaining release procedure is clean-org installation/testing followed by package-version promotion if the artifact passes validation.

30. Final reusable command sheet

```
# ===== ORG / DEV HUB =====
sf org list
sf org login web --alias FallingLily --set-default-dev-hub

# ===== PROJECT =====
sf project generate --name EnterpriseTaskManager

# ===== SCRATCH ORG =====
sf org create scratch --definition-file config/project-scratch-def.json   --alias DevOrg --duration-days 7 --target-dev-hub Falli

# ===== DEPLOY / TEST =====
sf project deploy start --source-dir force-app --target-org DevOrg   --test-level RunLocalTests
sf apex run test --target-org DevOrg --test-level RunLocalTests

# ===== RETRIEVE =====
sf project retrieve start --target-org DevOrg --source-dir force-app

# ===== UNLOCKED PACKAGE =====
sf package create --name "EnterpriseTaskManager"   --package-type Unlocked --path force-app   --target-dev-hub FallingLily --no-n

sf package list --target-dev-hub FallingLily --verbose

sf package version create --package "EnterpriseTaskManager"   --code-coverage --wait 30   --target-dev-hub FallingLily --tag v1.0

sf package version create list --target-dev-hub FallingLily
sf package version list --packages "EnterpriseTaskManager"   --target-dev-hub FallingLily

sf package install --package 04tXXXXXXXXXXXXXXX   --target-org PackageTest --wait 30

sf package version promote --package 04tXXXXXXXXXXXXXXX   --target-dev-hub FallingLily --no-prompt

# ===== MANAGED 2GP =====
sf package create --name "EnterpriseTaskManager"   --package-type Managed --path force-app   --target-dev-hub FallingLily

sf package version create --package "EnterpriseTaskManager"   --code-coverage --wait 30   --target-dev-hub FallingLily --tag v1.0

sf package install --package 04tXXXXXXXXXXXXXXX   --target-org PackageTest --wait 30

sf package version promote --package 04tXXXXXXXXXXXXXXX   --target-dev-hub FallingLily --no-prompt

# ===== LEGACY UNMANAGED 1GP =====
sf package1 version create   --package-id 033XXXXXXXXXXXXXX   --name "Version 1.0" --version 1.0   --target-org PackagingOrg --w

# ===== GIT =====
git init
git branch -M main
git add .
git commit -m "Initial Salesforce DX project"
git remote add origin git@github.com:ORG/REPO.git
git push -u origin main
```

31. Enterprise definition of done

Before declaring a package-ready enterprise application complete, confirm all of the following:

• The DX project is clean and reproducible from source control.

• Every required metadata component is present in the package directory.

• A fresh scratch org can be created from the repository.

• The source deploys successfully to that scratch org.

• All Apex tests pass without relying on pre-existing org data.

• The LWC can retrieve and mutate real Salesforce records.

• DML and SOQL are bulk-safe and appropriately secured.

• Required objects, fields, tabs, apps, flows, labels, permission sets, and configuration are packageable.

• No hard-coded Salesforce IDs or developer-only endpoints exist.

• Secrets are outside source control and supplied through secure configuration.

• The package installs successfully in a clean org.

• The installed app works without manual developer intervention.

• The package version is tagged to a Git commit.

• Release notes and upgrade instructions exist.

• A rollback/upgrade strategy has been considered.

• CI validates pull requests and release artifacts.

• The target package type matches the distribution goal: Unlocked for internal apps, Managed 2GP for customer/AppExchange distribution.

32. Official Salesforce references

The following Salesforce documentation was used to validate the packaging terminology and current workflow:

• Salesforce DX Project Structure and Source Format — https://developer.salesforce.com/docs/platform/sfdx-dev/guide/sfdx-dev-source-file-format.html

• Salesforce DX Project Setup — https://developer.salesforce.com/docs/platform/sfdx-dev/guide/sfdx-dev-workspace-setup.html

• Unlocked Packages — https://developer.salesforce.com/docs/platform/sfdx-dev/guide/sfdx-dev-unlocked-pkg-intro.html

• Workflow for Unlocked Packages — https://developer.salesforce.com/docs/platform/sfdx-dev/guide/sfdx-dev-unlocked-pkg-workflow.html

• Unlocked Package Best Practices — https://developer.salesforce.com/docs/platform/sfdx-dev/guide/sfdx-dev-unlocked-pkg-best-practices.html

• Understanding Namespaces — https://developer.salesforce.com/docs/platform/sfdx-dev/guide/sfdx-dev-unlocked-pkg-plan-namespaces.html

• Second-Generation Managed Package Workflow — https://developer.salesforce.com/docs/platform/pkg2-dev/guide/sfdx-dev-dev2gp-workflow.html

• Create a Second-Generation Managed Package — https://developer.salesforce.com/docs/platform/pkg2-dev/guide/sfdx-dev-dev2gp-create-pkg-base.html

• Create Versions of a Second-Generation Managed Package — https://developer.salesforce.com/docs/platform/pkg2-dev/guide/sfdx-dev-dev2gp-create-pkg-ver.html

• Salesforce CLI package commands — https://developer.salesforce.com/docs/platform/salesforce-cli-reference/guide/cli_reference_package.html

• Salesforce CLI package1 commands — https://developer.salesforce.com/docs/platform/salesforce-cli-reference/guide/cli_reference_package1.html

• First-Generation package version creation — https://developer.salesforce.com/docs/platform/salesforce-cli-reference/guide/cli_reference_package1_version_create.html

• Package installation — https://developer.salesforce.com/docs/platform/salesforce-cli-reference/guide/cli_reference_package_install.html

These links are included as a maintenance aid; Salesforce CLI syntax and product behavior can change, so verify command help in the installed CLI when starting a future project.